Programação e Algoritmos

Estruturas de controle

{.js}
JavaScript

node
JS®

Wagner Cesar Vieira
wagner.vieira01@cps.sp.gov.br

Agenda

- Sintaxe do Javascript
- Variáveis
- Tipos de dados
- Estruturas de decisão: if e else
- Estruturas de decisão: switch
- Estruturas de repetição: laço for
- **Exercícios**
- Estrutura de repetição: laço while
- Estrutura de repetição: laço do...while

Tipos de dados

JavaScript aceita os tipos:

- **number** (inteiros ou com decimais), **string**, **boolean** e **null**;

Entretanto apresenta tipagem fraca, não sendo necessário declarar o tipo de uma variável e a mesma poderá receber valores de um tipo diferente, posteriormente.

Exemplo

```
let x = 0;
x = "Olá!";
x = true;
x = null;
```

Nesse exemplo, a variável x é declarada inicialmente com um valor numérico e, sucessivamente, atribui-se a ela uma string, um boolean e, por fim, null.

Tipos de dados

Booleano

Uma variável do tipo booleano pode assumir apenas dois valores: true e false.

Os valores deste tipo são em geral usados pela linguagem como resultado de comparações e podem ser usados pelo usuário para valores de teste ou para atributos que possuam apenas dois estados.

O JavaScript converte automaticamente **true** para 1 e **false** para 0 quando isso for necessário.

```
let status = true;
console.log(status);
```

Tipos de dados

Indefinido (undefined)

Uma variável é indefinida quando ela foi declarada de alguma forma mas não possui nenhum valor concreto armazenado. Quando tentamos acessar uma variável que não teve nenhum valor associado a ela teremos como retorno "**undefined**" (indefinido).

```
let naosei;
console.log(naosei);
```

```
PS C:\Users\Wagner\Projetos\variaveis> node .\index.js
undefined
PS C:\Users\Wagner\Projetos\variaveis>
```

Tipos de dados

Null

O null é a ausência de valor; quando atribuímos null a um objeto ou variável significa que essa variável ou objeto não possui valor válido.
Para efeito de comparação, se usarmos o operador de igualdade "==", JavaScript irá considerar iguais os valores null e undefined.

No entanto, se for necessário diferenciar os dois valores é recomendável o uso do operador "===" de identicidade. Assim, para efeito de comparação, undefined e null são iguais, mas não idênticos.

```
let sera = null;
console.log(sera);
```

```
PS C:\Users\Wagner\Projetos\variaveis> node .\index.js
null
PS C:\Users\Wagner\Projetos\variaveis>
```

Strings

Strings são sequências de caracteres. Em JavaScript a string pode ser tanto um tipo **primitivo** de dado como um objeto; no entanto, ao manipulá-la temos a impressão de que sejam **objetos** pois as strings em JavaScript possuem **métodos** que podemos invocar para realizar determinadas operações sobre elas

```
let frase1 = 'Hoje é amanhã,'
let frase2 = " mas poderia ser ontem,"

let frase3 = new String(' se não fosse amanhã.')

console.log(frase1 + frase2 + frase3)
```

Entrada e saída de dados

Prompt-sync

O prompt-sync é uma biblioteca JavaScript para Node.js que permite solicitar entrada de dados do usuário diretamente no terminal de forma síncrona, bloqueando a execução do código até que uma resposta seja fornecida.

Como utilizar:

Instalação: Precisa ser instalado no projeto via npm: **npm install prompt-sync**.
Uso no código:

```
index.js > ...
1   const prompt = require('prompt-sync')();
2   const nome = prompt('Digite seu nome: ');
3   console.log(`Olá, ${nome}!`);
```

Entrada e saída de dados

Console.log()

O console.log() é uma ferramenta essencial para desenvolvedores JavaScript que permite a exibição de mensagens de log no console do navegador.

Esse método é amplamente utilizado para depuração de código, rastreamento de variáveis e monitoramento do fluxo de execução de um programa.

```
const num = prompt ( 'Digite um número: ' ) ;
console.log ( 'Seu número + 4 = ' ) ;
console.log ( Número ( num ) + 4 ) ;
```

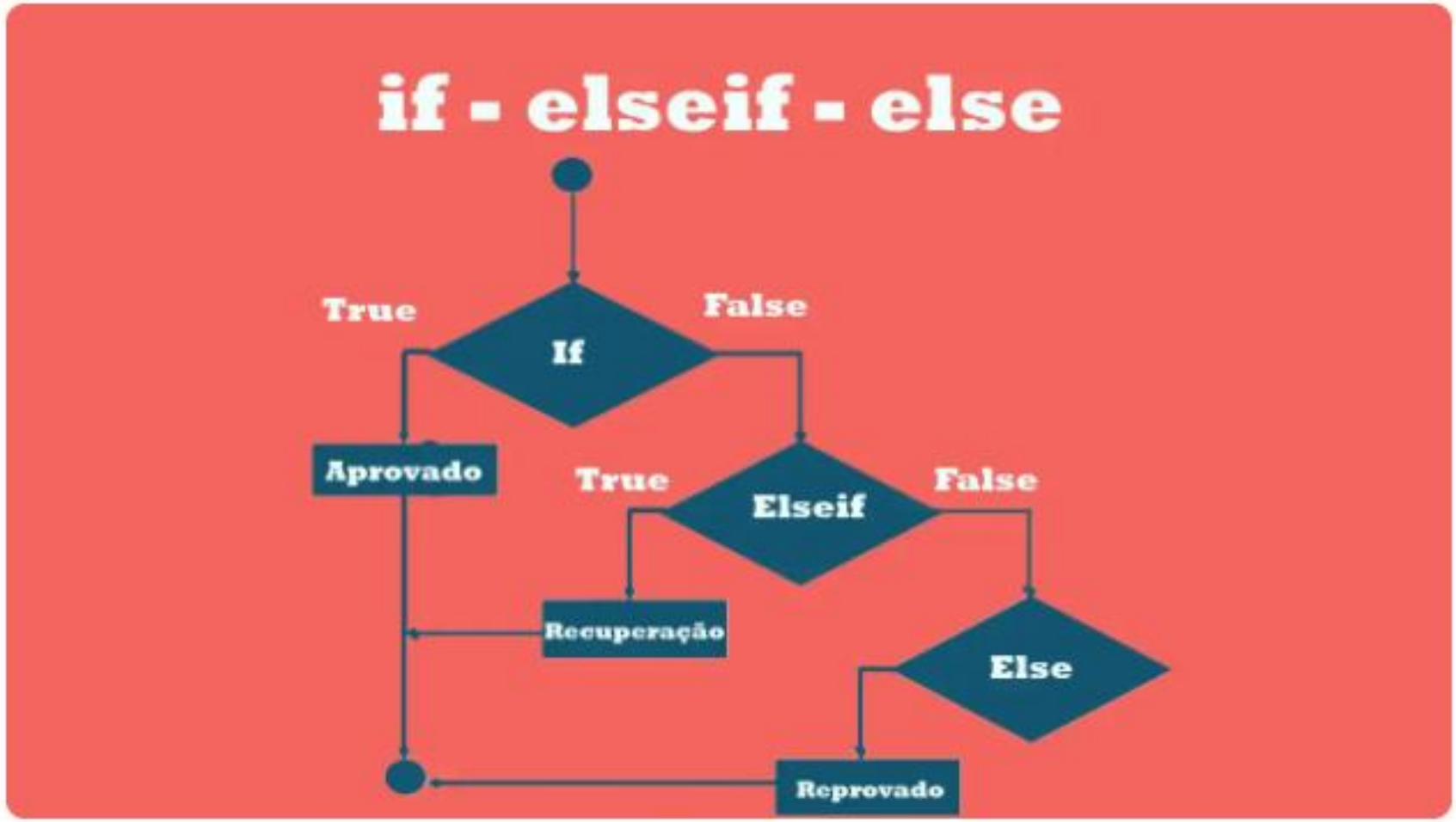
Estruturas de decisão: if e else

A condicional if é uma estrutura condicional que executa a afirmação, dentro do bloco, se determinada condição for verdadeira. Se for falsa, executa as afirmações dentro de else.

```
if (condicao1) {
    //instruções
}
else {
    //instruções
}
```

```
if (condicao1) {
    //instruções
}
else if (outracondicao) {
    //instruções
}
else {
    //instruções
}
```

Estruturas de decisão: if e else



Fonte: https://hermes.dio.me/assets/articles/cc199405-4742-470e-b9ed-9a68175107e1.jpg

Exemplo if e else


```
// Exemplo de if simples e composto
let semaforo = 'verde';

if (semaforo == 'verde') {
    let mensagem = 'Pode passar';
}
```

```
let semaforo = 'verde';

if (semaforo == 'verde') {
    let mensagem = 'Pode passar';
}
else {
    mensagem = 'Pare e aguarde';
}
```

Exemplo if aninhado


```
// Exemplo de if aninhado
let semaforo = 'vermelho';
let ligado = false;

if (ligado == true) {
    if (semaforo == 'verde') {
        mensagem = 'Pode passar';
    } else if (semaforo == 'vermelho') {
        mensagem = 'Pare';
    } else {
        mensagem = 'Atenção';
    }
}
else {
    mensagem = ('Aguarde no acostamento');
}

console.log(mensagem)
```

Exercícios de Javascript e Nodejs

1 – Crie um algoritmo Javascript com Nodejs (com comentários) para receber 3 notas do usuário, calcular a média e exibir o resultado.

2 – Elabore um algoritmo que receba um número qualquer e calcule e imprima o seu quadrado. Exemplo: digita 3 e retorna 9.

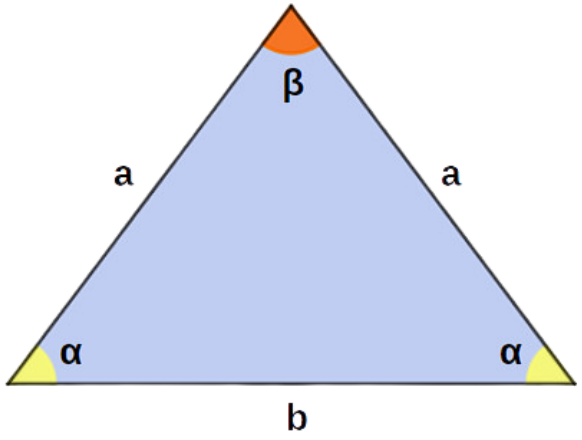
3 – Desenvolva um algoritmo que solicite 3 números inteiros ao usuário, verifique qual é o maior e o menor e exiba os resultados. Comente no código.

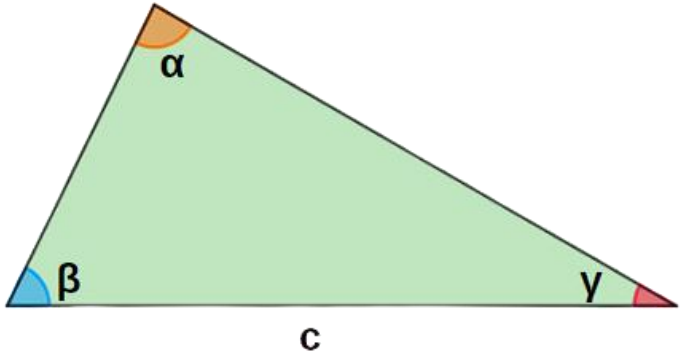
4 – Construa um algoritmo que solicita ao usuário que informe 4 idades. Depois, exiba quantas pessoas são maiores de idade (18 anos) e quantas são menores. Explique nos comentários.

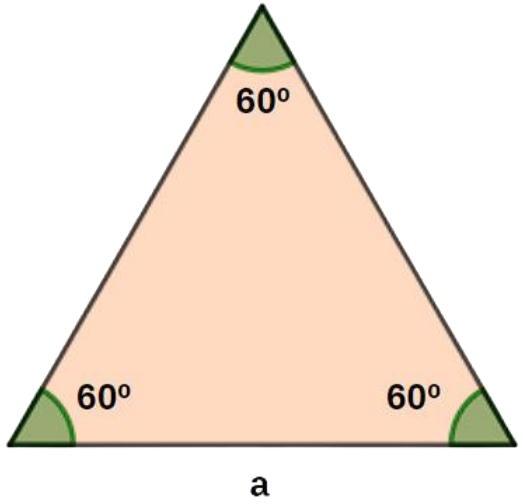
Desafio: triângulos

5. Desenvolva um algoritmo, em Javascript, que atenda às especificações de existência de triângulos, solicitando as 3 (três) medidas ao usuário e, classifique quanto aos lados.

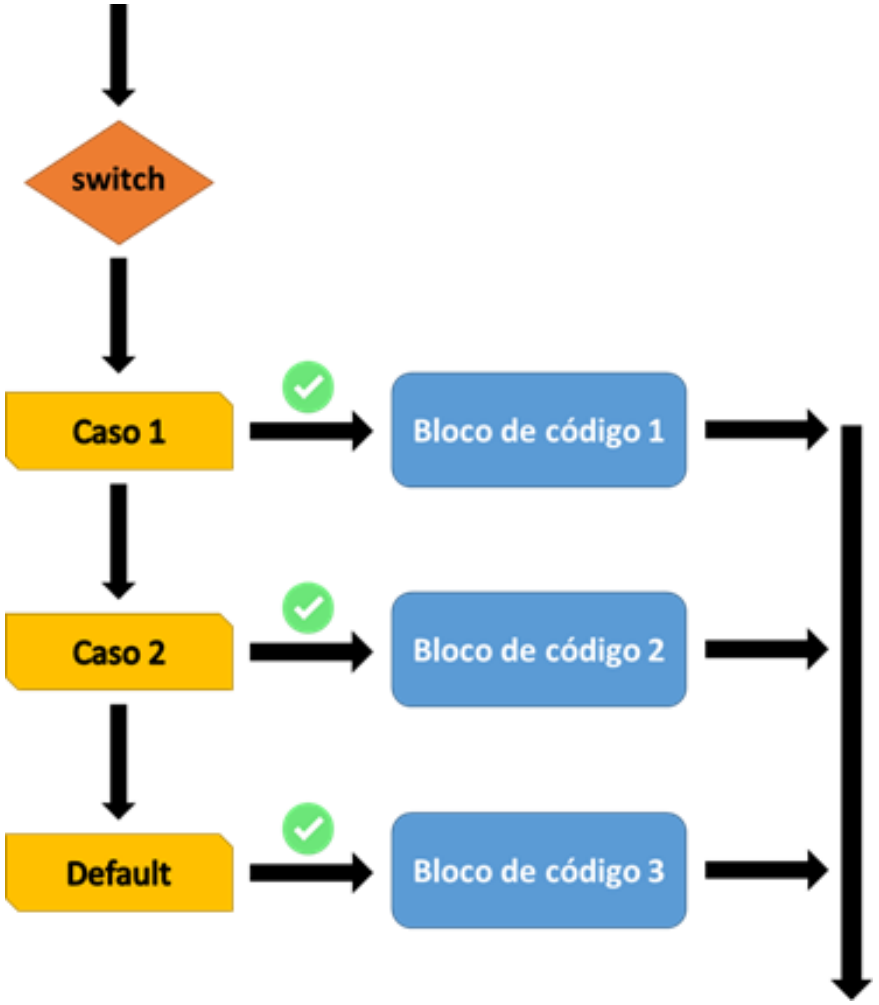
- ❑ Equilátero;
- ❑ Isósceles;
- ❑ Escaleno.







Switch




```
switch (expressao) {
   case valor 1:
            //instruções 1
           break;
   case valor 2:
            //instruções 2
           break;
   case valor n:
            //instruções 3
           break;
  default:
           //saída padrão
}
```

Exemplo de Switch

index.js > ...

```js
import promptSync from 'prompt-sync';

const prompt = promptSync();
const senha = parseInt(prompt('Digite a senha de 8 dígitos do cofre: '));

switch(senha) {
    case 12345678: console.log('Senha incorreta!!!'); break;
    case 87654321: console.log('Senha incorreta!!!'); break;
    case 13579135: console.log('Senha incorreta!!!'); break;
    case 24688642: console.log('Senha incorreta!!!'); break;
    case 11235813: console.log('Senha correta! Parabéns!'); break;
    default: console.log('Combinação errada. Desculpe'); break;
}
```

Exercícios com Switch

6 – Elabore um algoritmo Javascript com Nodejs (com comentários) para exibir o dia da semana de 1 (Domingo) a 7 (Sábado). Se for inválido, exiba "Dia inválido".

7 – Crie um algoritmo que receba uma letra e determine se ela é vogal ou consoante e exiba no console.

8 – Desenvolva um algoritmo que baseado no número do mês do ano (1 a 12), exiba a estação do ano. Comente o código.

9 – Construa um algoritmo que exiba um menu:
"1 – Cadastrar"
"2 – Listar"
"3 – Atualizar"
"4 – Excluir"
"5 – Sair"

Lista de tarefas

1. Criar repositório no GitHub;
2. Compreender e interpretar o problema;
3. Iniciar a codificação da solução;
4. No final das aulas, subir para o repositório.

Etec
Dr. Adail Nunes da Silva
Taquaritinga

1. Criar repositório no GitHub

1 General

Owner * wcrvieira / Repository name * algoritmos_js

algoritmos_js is available.

Great repository names are short and memorable. How about improved-bassoon?

Description

Algoritmos com Javascript e Nodejs

34 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository — Public

Add README
READMEs can be used as longer descriptions. About READMEs — On

Add .gitignore
.gitignore tells git which files not to track. About ignoring files — Node

Add license
Licenses explain how others can use your code. About licenses — No license

Create repository

Repositório do projeto

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

wcrvieira Iniciando os algoritmos Node e Javascrip ee49c78 · now 1 Commit

Estruturas condicionais.pdf	Iniciando os algoritmos Node e Javascrip	now
Iniciando com Javascript.pdf	Iniciando os algoritmos Node e Javascrip	now

README

Add a README

Help people interested in this repository understand your project.

Add a README

Comandos GIT

// Configura o usuário do GitHub – variável local

git config –global user.name “seu_usuário”

// Configura o e-mail do GitHub – variável local

git config –global user.email “seu_email_do_github”

// Configura o repositório do GitHub – criado anteriormente

git remote add origin “seu_repositório_git”

Comandos GIT

// Adiciona os arquivos do projetos no git local

git add .

// Confirma a inclusão dos arquivos para enviar

git commit –m “Sua mensagem inicial”

// Envia os arquivos do local para o remoto (GitHub)

git push –u origin **main ou master** --force

Comandos GIT

// Limpa o repositório remoto

```
git remote remove origin
```

// Verifica o status dos arquivos

```
git status
```

// Lista as configurações locais do Git

```
git config --list
```

Wagner Cesar Vieira
wagner.vieira01@cps.sp.gov.br